

Bootcamp: Full Stack Junior

Modulo 1: Gestión Integral de Bases de Datos

Semana 2

Contenido

1

Subconsultas

2

Vistas

3

Joins complejos

4

Procedimientos almacenados

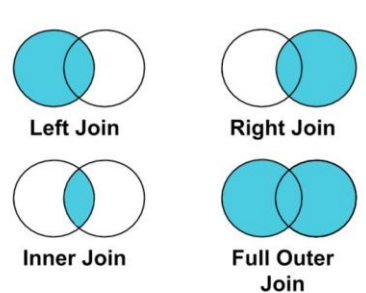
5

Triggers

6

Funciones SQL

Contenido:

Arquitecturas de Consulta y Automatización		
1. Relaciones y Estructuras de Consulta Avanzadas		
El dominio de las relaciones es el pilar de cualquier base de datos relacional. No se trata solo de unir tablas, sino de entender cómo fluye la información entre entidades para garantizar la integridad y la eficiencia.		
• Joins (Uniones): Son operaciones que permiten combinar filas de dos o más tablas basándose en una columna relacionada. ○ Inner Join: Devuelve registros con valores coincidentes en ambas tablas. ○ Left (Outer) Join: Devuelve todos los registros de la tabla izquierda y los coincidentes de la derecha. ○ Right (Outer) Join: Devuelve todos los registros de la tabla derecha y los coincidentes de la izquierda. ○ Full (Outer) Join: Devuelve todos los registros cuando hay una coincidencia en una de las tablas.		
SQL Joins  The diagram shows four Venn diagrams representing different SQL join types. Each diagram consists of two overlapping circles. In the 'Left Join' diagram, the left circle is shaded blue. In the 'Right Join' diagram, the right circle is shaded blue. In the 'Inner Join' diagram, only the intersection of the two circles is shaded blue. In the 'Full Outer Join' diagram, both the left and right circles are shaded blue.		
<i>Ilustración 1 - Tipos de Joins - Fuente: Medium.com</i>		
• Common Table Expressions (CTE): Una CTE es un conjunto de resultados temporal que se define dentro de la ejecución de una sola instrucción SQL (SELECT, INSERT, UPDATE o DELETE). Es fundamental para: ○ Legibilidad: Divide consultas masivas en bloques lógicos. ○ Recursividad: Permite manejar estructuras jerárquicas (como organigramas).		
Técnica	Propósito Principal	Cuando Usarla

Joins	Relacionar entidades físicas	En consultas estándar de reportes y cruces de datos.
CTE	Organizar lógica compleja	Cuando se requiere limpiar datos antes del
Subconsultas	Filtrado dinámico	Cuando el criterio de filtrado depende del resultado de otra consulta

Tabla 1 - Comparativa de Estructuras de Consulta - Fuente: Autor de Contenidos

2. Lógica de Subconsultas y Filtrado Dinámico

Las subconsultas (o queries anidadas) permiten que una consulta actúe como un parámetro de entrada para otra.

- **Subconsultas Escalares:** Devuelven un solo valor (útiles para comparaciones en el WHERE).
- **Subconsultas Correlacionadas:** Referencian columnas de la consulta externa, ejecutándose una vez por cada fila procesada.
- **Operadores Lógicos Clave:**
 - **IN / NOT IN:** Para verificar pertenencia a un conjunto.
 - **EXISTS / NOT EXISTS:** Más eficiente que IN para verificar presencia de registros sin cargar los datos.



Ilustración 2 - Lógica de subconsultas - Fuente: Gemini Nano Banana Pro

3. Automatización y Programación: Stored Procedures y Triggers

Para que una base de datos sea profesional y escalable, debe ser capaz de ejecutar lógica interna de forma autónoma.

- **Stored Procedures (Procedimientos Almacenados):** Son scripts SQL guardados en la base de datos que pueden aceptar parámetros, ejecutar lógica condicional y ser llamados por aplicaciones externas.
 - *Ventaja:* Reducen el tráfico de red y centralizan la lógica de negocio.
- **Triggers (Disparadores):** Programas que se ejecutan automáticamente ("se disparan") ante eventos específicos como INSERT, UPDATE o DELETE en una tabla.
 - *Uso común:* Auditoría de cambios y validaciones complejas de integridad.

Característica	Stored Procedure	Trigger
Ejecución	Manual / Por la aplicación	Automática por evento
Parámetros	Permite entrada y salida	No permite parámetros
Finalidad	Automatizar procesos recurrentes	Mantener integridad y auditoría

Tabla 2 - Diferencias entre Procedimientos y Triggers - Fuente: Autor de Contenidos

4. Automatización y Programación: Más allá de las Consultas Simples

La automatización en SQL busca el principio **DRY (Don't Repeat Yourself)**. En lugar de escribir la misma lógica de cálculo o validación en múltiples partes de una aplicación, la centralizamos en la base de datos.

4.1. Common Table Expressions (CTE) y Legibilidad

Una CTE permite definir un conjunto de resultados temporal que existe solo durante la ejecución de una consulta. A diferencia de las subconsultas tradicionales, las CTE se definen al inicio, lo que facilita la depuración y la creación de consultas jerárquicas.

- **Estructura Jerárquica:** Las CTE recursivas permiten navegar por tablas que tienen relaciones consigo mismas (ej. una tabla de empleados donde cada uno tiene un jefe que también es un empleado).
- **Modularización:** Puedes definir múltiples CTE en una sola sentencia, permitiendo que una CTE "B" consuma los resultados de una CTE "A".

- **Optimización de Código:** Evitan la redundancia de cálculos complejos que deben repetirse en el SELECT y el WHERE.

Ejemplo de sintaxis de código:

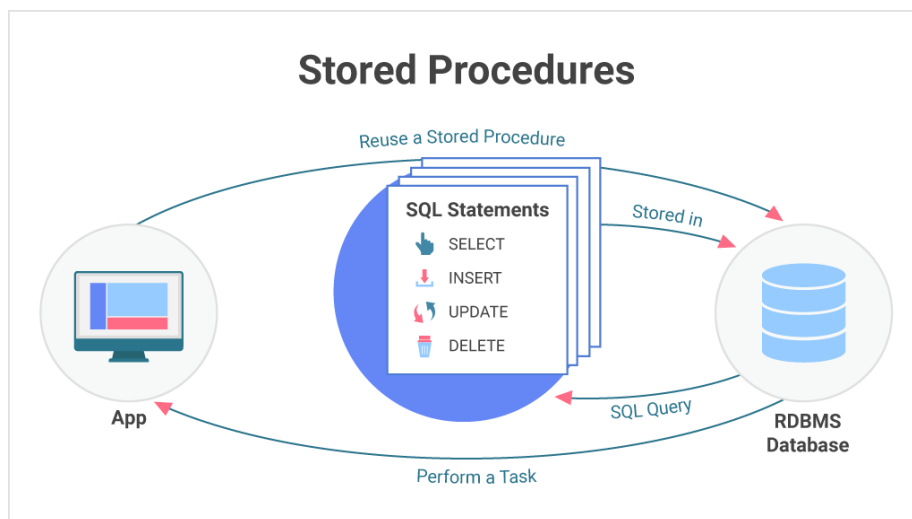
```
WITH ResumenVentas AS (  
  SELECT id_producto, SUM(cantidad) AS total_vendido  
  FROM Ventas  
  GROUP BY id_producto  
)  
SELECT p.nombre, r.total_vendido  
FROM Productos p  
JOIN ResumenVentas r ON p.id_producto = r.id_producto;
```

Ilustración 3 - Ejemplo sintaxis CTE - Fuente: Autor de Contenidos

4.2. Stored Procedures (Procedimientos Almacenados)

Los procedimientos almacenados son bloques de código SQL que se compilan y guardan en el servidor. Son el equivalente a las "funciones" en lenguajes como JavaScript, pero optimizados para el manejo de datos.

- **Parámetros de Entrada y Salida:** Permiten recibir variables del usuario y devolver resultados o valores de estado.
- **Lógica de Control de Flujo:** Soportan sentencias IF . . . ELSE, CASE, y bucles como WHILE, permitiendo tomar decisiones complejas antes de insertar o modificar datos.
- **Seguridad Mejorada:** Al usar procedimientos, se pueden otorgar permisos para ejecutar el proceso sin dar acceso directo a las tablas, lo que previene ataques de **SQL Injection**.



Ejemplo de sintaxis de código:

```
DELIMITER //
CREATE PROCEDURE sp_RegistrarVenta(IN p_id INT, IN p_cant INT)
BEGIN
    -- Lógica de negocio
    INSERT INTO Ventas(id_producto, cantidad, fecha)
    VALUES (p_id, p_cant, NOW());
END //
DELIMITER ;

-- Para ejecutarlo:
CALL sp_RegistrarVenta(101, 2);
```

Ilustración 5 - Ejemplo sintaxis Procedimiento almacenado - Fuente: Autor de Contenidos

4.3. Triggers (Disparadores) y la Integridad de los Datos

Un Trigger es un objeto que "reacciona" ante un evento de manipulación de datos (INSERT, UPDATE o DELETE). Son herramientas críticas para la seguridad y la auditoría.

- **Tipos de Disparadores:**
 - **BEFORE:** Se ejecuta antes de que el cambio se guarde (útil para validaciones o formateo automático de datos).
 - **AFTER:** Se ejecuta después del cambio (ideal para registrar logs de auditoría o actualizar tablas de estadísticas).
- **Tablas Virtuales (OLD y NEW):** Los triggers permiten comparar los valores antiguos con los nuevos, lo que facilita detectar si un cambio de precio, por ejemplo, excede un porcentaje permitido.

Ejemplo de sintaxis de código:

```
CREATE TRIGGER tr_ActualizarStock
AFTER INSERT ON Ventas
FOR EACH ROW
BEGIN
    -- Resta la cantidad vendida del inventario automáticamente
    UPDATE Productos
    SET stock = stock - NEW.cantidad
    WHERE id_producto = NEW.id_producto;
END;
```

Ilustración 6 - Ejemplo sintaxis Trigger - Fuente: Autor de Contenidos

Herramienta	Cuando se ejecuta	Persistencia	Uso Principal
-------------	-------------------	--------------	---------------

CTE	Solo durante la ejecución de la consulta	No se guarda en disco	Limpiar y organizar reportes complejos.
Stored Procedure	Cuando el usuario o la app lo llama	Permanente en el servidor	Procesos de negocio (ej. realizar un cierre de caja).
Trigger	Automáticamente ante un evento de datos	Permanente en el servidor	Auditoría y reglas de integridad que nunca deben romperse.

Tabla 3 - Comparativa de Herramientas de Automatización - Fuente: Autor de Contenidos

Recursos de Apoyo



Material de Lectura

[Creación de procedimientos almacenados.](#)



Guia

[Guia de ejemplo de cómo crear CTe's](#)



Material de Lectura

[Joins MySQL.](#)



Material de Lectura

[Uso de triggers en MySQL](#)